1. Design Class: User

Description: 'Model' component containing user information as its attributes, using setter/getter operations to create/update or retrieve its attribute. This model is mapped to an equivalent database table.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userId	Integer	null	- randomly auto-generated - exclusive
2	username	String	null	exclusively exists to identify users using the software.
3	email	String	null	saved when using gmail for registration
4	password	String	null	must be hashed before stored into database
5	role	String	Product Manager	Administrator / Product Manager
6	status	Boolean	True	True as Unblocked, False as Blocked
7	province	String	null	select from a list of city
8	district	String	null	select from a list of district
9	address	String	null	detailed address information

Table 2 . Operation design

#	Name	Return type	Description
1	getUserId	Integer	retrieve 'id' attribute
2	getUsername	String	retrieve 'username' attribute
3	getEmail	String	retrieve 'email' attribute
4	getPassword	String	retrieve 'password' attribute
5	getRole	String	retrieve 'role' attribute
6	getStatus	Boolean	update 'status' attribute
7	getDistrict	String	retrieve 'status' attribute
8	getProvince	String	retrieve 'city' attribute
9	getAddress	String	retrieve 'address' attribute

10	User	constructor	constructor with all mandatory attributes

Parameter:

- **setter operations** have only a parameter with data type as the corresponding attribute's data type.
- **getter operations** have no parameters.
- **User(userId: Integer, username: String, email: String, password: String, role: String, status: Boolean, province: String, district: String, address: String)**

Exception

- all **setter operations** can throw a **MandatoryFieldException** which avoids blanking field.

Method:

- **setter methods** are implemented by assigning the corresponding attribute to a specific value.
- **getter methods** are implemented by returning the value of the corresponding attribute.

State:

- if any parameter in **setter operations** is blank, the method will throw **'MandatoryFieldException'.**

2. Design Class: UserService

Description: Service component managing user information in related-business logic.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	users	List<User>	null	list of all users from database
2	currentUser	User	null	current user

Table 2. Example of operation design

#	Name	Return	Description
1	getAllUsers	List<User>	fetch all User instances
2	getUserByUserId	User	fetch User instance by 'userId'

3	getUserByUsername	User	fetch User instance by ‘username’
4	checkExistenceOfUsername	Boolean	check if the provided username exists in AIMS system
5	checkSecurityOfPassword	Boolean	check constraints of password to secure password.
6	checkExistenceOfEmail	Boolean	check if the provided email address exists.
6	saveUser	void	save a specific user into database

Parameter:

- getAllUser(): void
    - **Return:** true if the available inventory is sufficient, otherwise false.
- getUserByUserId(usedId: Integer): User
    - **userId**: An integer representing the unique identifier of the user.
    - **Return**: a User instance with a user id equaling the given parameter.
- checkExistenceOfUsername(username: String): Boolean
    - **username**: a String representing username of the user, entered when create/update user procedures
    - **Return**: return true if username exists on database and false if username does not exist.
- checkSecurityOfPassword(password: String): Boolean
    - **password**: a String representing the password of the user entered when create/update user procedures, need to use the hash algorithm before comparison.
    - **Return**: return true if password matches security constraints, false if not.
- checkExistenceOfEmail(email: String): Boolean
    - **password**: a String representing the email of the user entered when create/update user procedures
    - **Return**: return true if email exists, false if not.
- saveUser(user: User): void
    - **user**: an User instance which contains user information provided by ‘Administrator.
    - **Return**: void.

Method

- getAllUser(): void
    - **Purpose**: to fetch all users to update its attribute ‘users’ after each time database updates.
    - **Usage**: connect to database then use fetch command of used DBMS.
- getUserByUserId(usedId: Integer): User
    - **Purpose**: to fetch a specific User which is used to display and validate information.
    - **Usage**: connect to DBMS before executing the retrieve command.
- checkIfUsernameExists(username: String): Boolean

    - **Purpose**: used in login, create or update with user information.
    - **Usage**: compare with information contained in User instance.
- checkSecurityOfPassword(password: String): Boolean
    - **Purpose**: secure password
    - **Usage**: use several standard password constraints.
- saveUser(user: User): void
    - **Purpose**: used in creating/ updating user information.
    - **Usage**: connect to DBMS before executing update command.

Exception:

- getUserByUserId, getUserByUsername: throwing **UserNotFoundException** if there is no User with the given 'userId'.

State: no specific information.

3. Design Class: UserController

Description: Controller component managing UI display following the related events.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	userService	UserService	null	- an interface providing services to manage related-business logic

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	submitUserInformation	void	- triggered when a user clicks Button to finish entering a user information form.
2	validateUserInformation	Boolean	- subroutine of 'submitUserInformation' - validate user information - display notification on invalid fields
3	notifyUserViaEmail	boolean	- subroutine of 'submitUserInformation' - send email notification to users via EmailManager's services.
4	changeAccountStatus	void	- triggered when a user toggle the status Button on a specific user

Parameter:

- submitUserInformation(user: User):
    - **user**: A User instance containing new user information.

    - **Return**: void.
- validateUserInformation(user: User): Boolean
    - **user**: A User instance containing new user information.
    - **Return**: true if user information is validated, otherwise, notify user to enter again.
- notifyUserViaEmail(user: User): void
    - **user**: A User instance containing new user information.
    - **Return**: void.
- changeAccountStatus(user: User): void
    - **user**: A User instance containing new user information.
    - **Return**: void.

Exception: no specific information

Method:

- submitUserInformation(user: User): void
    - **Purpose**: wrap all subroutines from verification and notification after successful creation/ update.
    - **Usage**: called when a user finishes entering user information form
- validateUserInformation(user: User): Boolean
    - **Purpose**: verify all provided information
    - **Usage**: using checking services from its 'UserService' attribute.
- notifyUserViaEmail(user: User): void
    - **Purpose**: provide user information to log into the AIMS system.
    - **Usage**: using EmailManager to send a notification to a user.
- changeAccountStatus(): void
    - **Purpose**: modify status of a user's account from blocked to unblocked and from unblocked to blocked.
    - **Usage**: be called when you toggle Button 'status'

State:

- submitUserInformation(user: User): void
    - if 'validateUserInformation' succeeds, go through and 'notifyUserViaEmail'.

4. Design Class: DashboardView

Description: View component managing UI components and listening to events to perform action provided by Controller component.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	tableView	javafx.scene.control.TableView<User>	null	display a list of all users

2	root	javafx.scene.layout.BorderPane	null	root UI component
3	userControll er	UserController	null	corresponding Controller

Table 2. Example of operation design

#	Name	Return	Description
1	createTopNav	void	create navigation bar on the top of dashboard interface
2	createSideNav	void	create navigation bar on the left side of dashboard interface
3	createUserTableView	void	create a table containing a list of all users.
4	createUserInformationForm	void	create a modal display a user information form

Parameter:

- createTopNav(): void
- createSideNav(): void
- createUserTableView(): void
- createUserInformationForm(user: User): void

Exception: no specific information

Method:

- createTopNav(): void
  - **Purpose**: initialize top navigation bar including several options.
  - **Usage**: called inside class constructor.
- createSideNav(): void
  - **Purpose**: initialize side navigation bar.
  - **Usage**: called inside class constructor.
- createUserTableView(): void
  - **Purpose**: create a recyclable stage to navigate between multiple contents.
  - **Usage**: called inside class constructor.
- createUserInformationForm(user: User): void
  - **Purpose**: create UI component to display user information form require user to enter all mandatory information.
  - **Usage**: be called when a user clicks Button to create a new user or update a specific user.

State: no specific information

5. Design Class: UserInformationForm

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	saveButton	javafx.scene.control.Button	null	button to save entered user information
2	statusSwitch	javafx.scene.control.RadioButton	null	switch used to modify account status
2	labels	List<javafx.scene.control.Label>	null	“Username”, “Password”, …
3	textFields	List<javafx.scene.control.TextField>	null	input components

Table 2 . Operation design

#	Name	Return	Description
1	UserInformationForm	constructor	Constructor which initializes resource to render and handle UserInformationForm modal

Parameter:

- UserInformationForm(user: User, userController: UserController)

Exception: no specific information

Method:

- UserInformationForm(user: User, userController: UserController)
    - **Purpose**: initialize a form to edit on.
    - **Usage**: called when you click edit/ create button on ‘DashboardView’ component.

State: no specific information

6. Design Class: DatabaseManager

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	URL	String	null	url to connect to database system
2	USER	String	null	username to connect to database system
3	PASSWORD	String	null	password to connect database system

Table 2 . Operation design

#	Name	Return	Description
1	getConnection	java.sql.Connection	get connection before command execution

Parameter:

- getConnection(): java.sql.Connection

Exception:

- getConnection(): java.sql.Connection
    - throw java.sql.SQLException when can not connect to the database system, ensure configuration information and database server activated.

Method:

- getConnection(): java.sql.Connection
    - **Purpose**: used to establish connection between the AIMS application with the database system.
    - **Usage**: called when you need to execute a command in the database system.

State:

- getConnection(): java.sql.Connection
    - using ‘try-catch’ mechanism to avoid system corruption.

8. Design Class: EmailManager

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	STMPHost	String	null	SMTP host
2	STMPPort	String	null	port to send email

3	senderEmail	String	null	email address of sender
4	password	String	null	password equivalent email address of sender

Table 2 . Operation design

#	Name	Return	Description
1	isEmailValid	Boolean	check email format
2	sendEmail	Boolean	send email by this method

Parameter:

- isEmailValid(email: String): Boolean
    - **email**: email address of sender
    - **Return**: true if email format is matched, otherwise false
- send(recipient: String, subject: String, messageBody: String): void
    - **email**: email address of sender
    - **Return**: void

Exception:

- send(recipient: String, subject: String, messageBody: String): void
    - throw jakarta.mail.MessagingException when a user can not send email.

Method:

- send(recipient: String, subject: String, messageBody: String): void
    - **Purpose**: verify email address and send success confirmation when creating/ updating user information.
    - **Usage**: called when notifying the updated user or created user from Controller components.

State:

- send(recipient: String, subject: String, messageBody: String): void
    - using ‘isEmailValid’ to check before STMP connection establishment
    - using ‘try-catch’ mechanism to avoid system corruption.